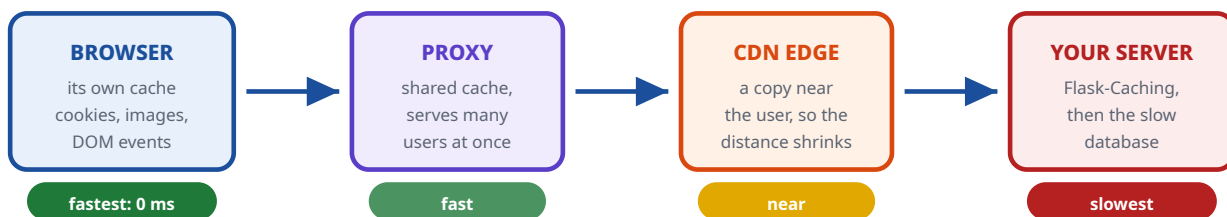


MODERN APPLICATION DEVELOPMENT II

Week 11 – Graded Assignment Complete Illustrated Solutions

Performance • Caching • ETag • Flask-Caching • Cookies • DOM events • Images • CDN

The journey of one request – and every place we can make it faster



THE GOLDEN RULE: stop the journey as early as you can. The best request is the one never sent.

Every question in this paper is about one box above, or about the arrows between them.

Caching decides where you stop. ETag decides if you can reuse. CDN decides how far you travel.

Written for a complete beginner. Start at Part 1 and read straight through. Every idea gets a picture and an everyday story before any code appears.

READ THIS FIRST

How to use this book

This document does **two** jobs at once:

1. **It teaches the subject from zero.** Parts 1 to 9 build the knowledge brick by brick, with pictures and everyday stories.
2. **It solves all 10 questions**, showing exactly how a marker expects you to reason.

GOOD NEWS ABOUT YOUR ATTEMPT

Your screenshots show **"Flag for review"** and **"Clear Selection"** and no green correctness banners, so these are the options you had *selected*, not confirmed results. Every answer here was therefore worked out independently and checked against the official documentation of Flask-Caching and the HTTP specification.

The result: **all ten of your selections are correct.** There is nothing to change. This book is here to make sure you know *why* – because the same ideas come back in the end-term paper with the wording shuffled.

Map of the book

Part	What you learn	Questions
Part 1	What "performance" actually means: latency, throughput, scalability	Q4
Part 2	The tools that measure it, and one famous tool that does not	Q5
Part 3	Caching, and the four places a cache can live	Q2, Q3
Part 4	ETag, If-Match and If-None-Match: reusing without re-downloading	Q9
Part 5	Flask-Caching: <code>@cached</code> vs <code>@memoize</code>	Q1
Part 6	Cookies: the hidden weight on every single request	Q6
Part 7	<code>DOMContentLoaded</code> vs <code>load</code> : when is the page ready?	Q7
Part 8	Images: the biggest easy win on most web pages	Q8
Part 9	CDN: putting a copy of your site near every user	Q10
Part 10	Answer key, cheat sheet, practice questions, glossary	revision

The boxes you will see

STORY BOX

A real-life picture of the idea. Read this one first, always.

DEFINITION BOX

The proper technical wording, the one to write in an exam.

TRAP BOX

The exact mistake that loses the mark. Learn these by heart.

ANSWER BOX

The final answer, with the option letters.

What "performance" actually means

People say "the site is slow" as if slowness were one thing. It is not. There are **three** separate measurements, and mixing them up is the commonest mistake in this topic.

1.1 Latency and throughput are different things

STORY BOX • THE WATER PIPE

Two questions you can ask about a pipe:

"How long does one drop take to travel from one end to the other?" That is **latency** – a *time*, measured in milliseconds. Lower is better.

"How many litres come out per minute?" That is **throughput** – an *amount per unit time*, measured in requests per second. Higher is better.

A thin, very long pipe has bad latency. A short but narrow pipe has bad throughput. They are independent, and a good application needs **both**: many requests handled per second, each one answered quickly.

The two numbers you must never confuse

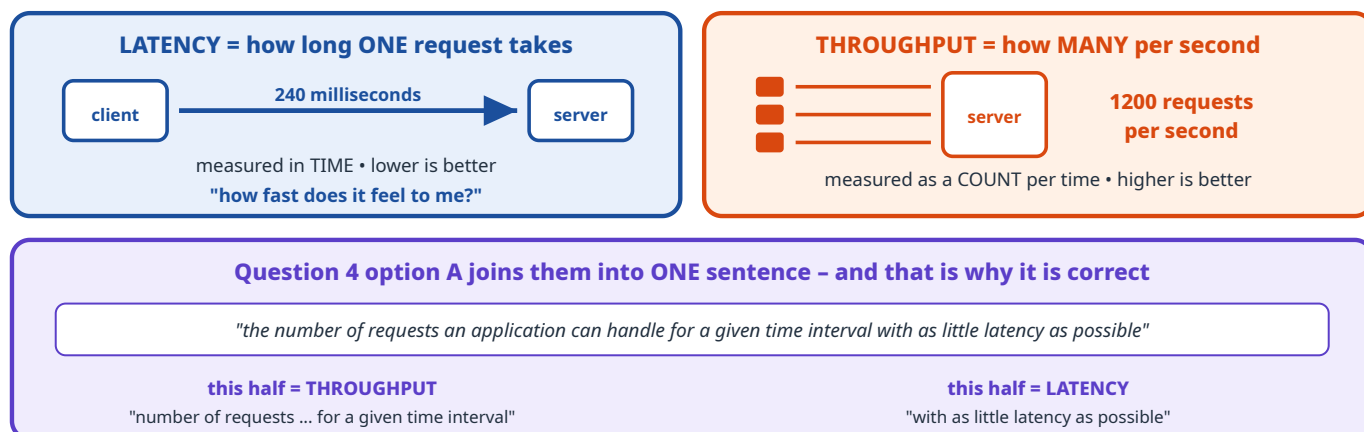


Figure 1.1 – Latency is a duration; throughput is a rate. Option A of Q4 asks for both at once.

1.2 The third measurement: scalability

STORY BOX • THE ONE-WINDOW TICKET COUNTER

A ticket counter serves one customer in 30 seconds. Lovely – when there is one customer. Now 200 people arrive at once. The clerk still takes 30 seconds each, so the last person waits 100 minutes.

Nothing about the clerk got slower. What failed was **scalability**: the ability to keep performing as the **load** grows. Measuring performance with one user tells you almost nothing about a real launch day.

How does the application behave as load increases?

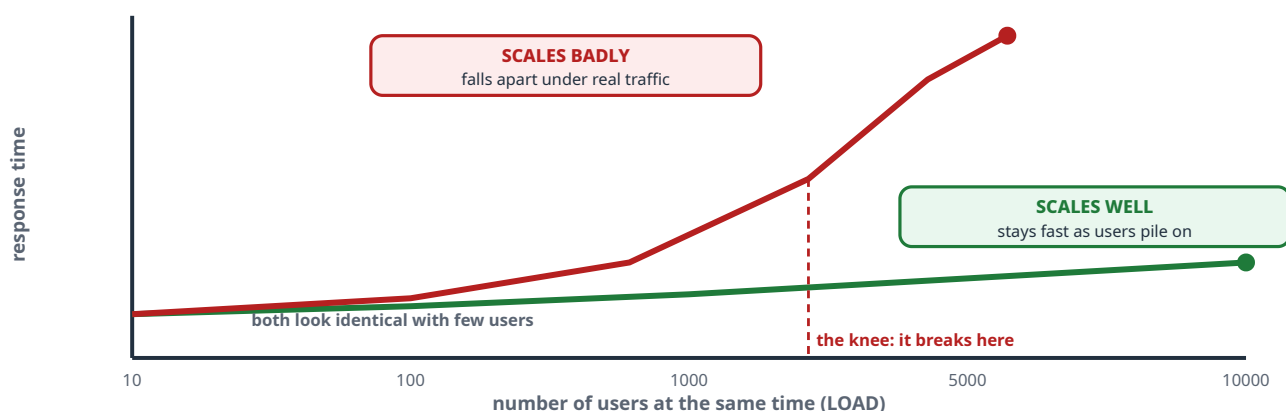


Figure 1.2 – Two applications that seem equally fast in testing and behave completely differently in production.

DEFINITION BOX • THE THREE MEASUREMENTS

- **Latency**: the time taken to serve one request. A duration. Lower is better.
- **Throughput**: how many requests are served per unit time. A rate. Higher is better.
- **Scalability**: whether latency and throughput hold up as the load grows.

All three are **measurable numbers**. That word matters for the question below.

1.3 What is NOT a performance criterion

STORY BOX • JUDGING A CAR BY ITS BADGE

You want to know how fast a car is. So you measure its top speed, its acceleration, and how it behaves fully loaded with luggage. What you do **not** do is say "it is a Maruti, therefore it is fast". The **brand is not a measurement**.

In the same way, "the app should be written in Flask or Django" is a statement about the **tool chosen**, not a measurement of speed. A brilliantly written Flask app and a terrible one are both Flask apps.

Is it a MEASUREMENT, or just a CHOICE?



Figure 1.3 – The test for any option in this question: could I attach a number to it?

Question 4

1 mark • select all that apply

Which of the following criteria should be directly considered while measuring the performance of an application?

- A. The number of requests an application can handle for a given time interval with as much as little latency.
- B. How does the application scale over load.
- C. The application should be written in Python based flask or Django framework.
- D. All of the above

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	This is throughput and latency in one sentence – the two headline numbers of performance. Figure 1.1 breaks the sentence in half to show both.
B	TRUE	Scalability. An application that is fast for one user and collapses at a thousand has not performed well. Figure 1.2.
C	FALSE	A technology choice, not a measurement. Performance is measured in numbers, and no number appears here. Plenty of fast applications use neither Flask nor Django, and plenty of slow ones use both.
D	FALSE	"All of the above" would require C to be true as well. Since C is false, D collapses with it.

ANSWER TO QUESTION 4

A and B

Throughput-with-latency, and scalability under load. Matches your selection.

TRAP BOX • THE FLATTERING OPTION

Option C mentions the exact framework you have been studying all term, which makes it feel like the "course-approved" answer. Examiners rely on that comfort. Apply the test from Figure 1.3 instead: **if you cannot attach a number to it, it is not a performance criterion.**

The tools that measure performance

STORY BOX • THE DOCTOR'S INSTRUMENTS

A thermometer measures temperature. A stethoscope listens to your heart. A weighing machine gives your weight. All three are medical instruments – but if the doctor wants your temperature, only one of them will do.

Lighthouse and **GTmetrix** are the thermometers of web performance. **Postman** is also a fine instrument, but it is for a different job entirely.

Performance tools, and one tool that measures something else

LIGHTHOUSE

built into Chrome DevTools

- audits performance, accessibility, SEO and best practices
- gives a score out of 100
- reports First Contentful Paint, Largest Contentful Paint, blocking time
- suggests concrete fixes

GTMETRIX

an online testing service

- loads your page from chosen locations around the world
- gives page load time and page size
- draws a waterfall chart of every single request the page made
- tracks scores over time

POSTMAN

an API development tool

- sends individual API requests
- checks status codes and JSON bodies are correct
- organises request collections
- used for TESTING CORRECTNESS, not analysing page performance

The honest nuance, so nobody catches you out

Postman does display the response time of a request, and modern versions can run load tests. But it is not a tool for measuring or analysing the performance of a WEB PAGE: it never renders the page, **so it cannot see images, CSS, JavaScript, the DOM, or anything the browser actually does.**

Figure 2.1 – Two page-performance tools and one API-correctness tool.

Question 5

1 mark • select all that apply

Which of the following tools can be used to measure or analyze the performance of a web application?

A. GTmetrix

B. Postman

C. Lighthouse

D. All of the above

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	GTmetrix exists for exactly this: it loads your page, times it, and produces a waterfall chart plus a list of recommendations.
B	FALSE	Postman is for developing and testing APIs – is the endpoint returning the right status code and the right JSON? It does not render a web page, so it cannot analyse page performance.
C	TRUE	Lighthouse is Google's auditing tool, built into Chrome DevTools. Performance is its headline score.
D	FALSE	Requires B, which is false.

ANSWER TO QUESTION 5

A and C

GTmetrix and Lighthouse. Matches your selection.

WORTH KNOWING FOR THE END TERM

Other genuine web-performance tools: **WebPageTest**, **PageSpeed Insights** (which runs Lighthouse), the **Network** and **Performance** tabs of Chrome DevTools, and for server-side load testing **Apache JMeter** or **Locust**. Tools that are *not* for page performance: Postman and `curl` (API calls), Git (version control), Selenium (functional testing).

Caching, and where caches live

STORY BOX • THE WATER BOTTLE ON YOUR DESK

Every time you are thirsty you could walk to the kitchen tap. Instead you fill a bottle once and keep it on your desk. Now drinking takes one second instead of one minute.

That bottle is a **cache**: a nearby copy of something that lives far away, kept so you do not have to make the long trip again. The trade-off is the same one every cache has: the water in your bottle might be **stale** while the tap is always fresh. Speed is bought with the risk of staleness.

DEFINITION BOX • CACHE

A **cache** stores a copy of a resource closer to the consumer so future requests are served faster. A request answered from the cache is a **cache hit**; one that has to travel to the original source is a **cache miss**. The original source is called the **origin**.

3.1 The four places a cache can live

The cache ladder: each rung you climb costs more time

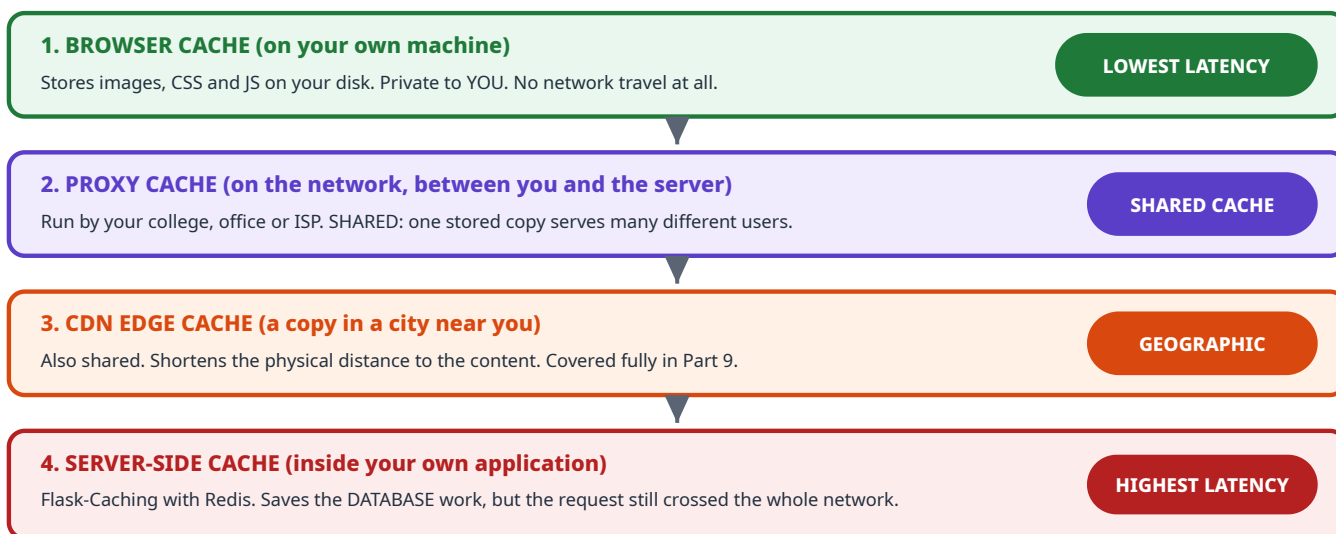


Figure 3.1 – Caching happens at **many** levels, not one. Note that browser level is nearest, so it has the least latency.

WHY BROWSER LEVEL HAS THE LEAST LATENCY

A browser cache hit involves **no network at all** – the file is read from your own disk. A proxy cache hit still requires a round trip out of your machine and onto the network, even if it is a short one. Distance is time, and zero distance wins.

3.2 Private cache vs shared cache

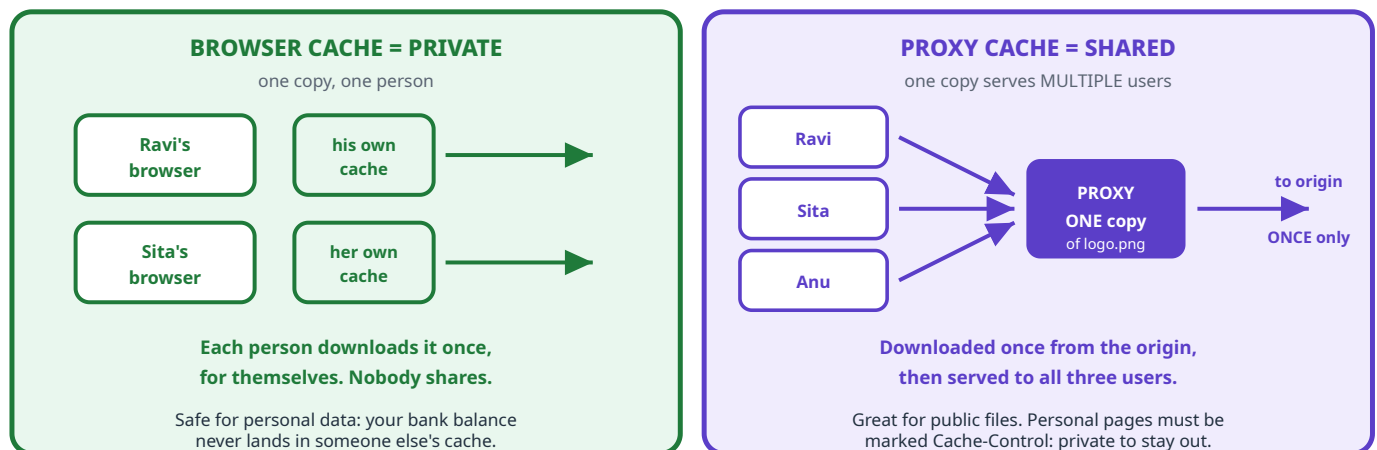


Figure 3.2 – The defining difference: a proxy cache serves its stored copy to **multiple users**.

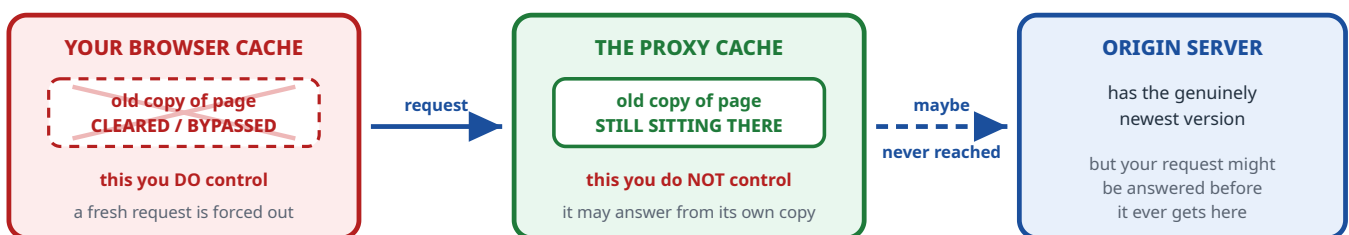
3.3 What a hard refresh really does

STORY BOX • EMPTYING YOUR OWN BOTTLE

You suspect the water in your desk bottle has gone stale, so you pour it away and refill it from the tap. Good. But your **neighbour's bottle** is still full of the old water, and the office **water cooler** in the corridor may also still hold yesterday's supply.

You control your own bottle only. **A hard refresh empties your browser's cache for that page – and has no authority over the proxy in the middle.**

You press Ctrl+Shift+R (hard refresh). What actually happens?



THE TWO FACTS THIS FIGURE GIVES YOU

Q2 option C: hard refresh DOES clear the browser cache for that page. TRUE

Q3 option B: a hard refresh CAN still be answered by a proxy's cached copy. TRUE

They are not contradictory: the first is about your machine, the second about the network beyond it.

Figure 3.3 – Your refresh button reaches only as far as your own computer.

Question 2

1 mark • select all that apply

Which of the following statement(s) is/are correct regarding caching?

- A. Caching can be done at browser level.
- B. Caching cannot be done at server (proxy) level.
- C. Hard refreshing a web page clears the browser cache for that specific web page.
- D. All of the above

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	The browser cache is rung 1 of Figure 3.1 – the most common cache of all. Your browser is storing images and stylesheets from this course's website right now.
B	FALSE	The word cannot again. Caching at proxy and server level is completely normal: proxy caches, CDN edge caches, and application caches such as Flask-Caching with Redis. Rungs 2, 3 and 4 of Figure 3.1.
C	TRUE	A hard refresh (<code>Ctrl+Shift+R</code> , or <code>Cmd+Shift+R</code> on a Mac) discards the browser's cached copy for that page and forces a fresh download. Left panel of Figure 3.3.
D	FALSE	Requires B to be true. It is not.

ANSWER TO QUESTION 2

A and C

Matches your selection.

Question 3

1 mark • single correct

Which of the following statement(s) is true?

- A. A cached resource stored at a proxy server serves its copy to multiple users.
- B. Hard refreshing a web page can still result in a cached response being served from a proxy server.
- C. Caching at browser level provides the least latency, if compared with proxy level.
- D. All of the above

A RARE CASE: "ALL OF THE ABOVE" IS THE ANSWER

In Q2, Q4 and Q5 the "All of the above" option was a trap. Here it is genuinely correct. That is exactly why you must **check every option every time** rather than following a habit. Examiners alternate deliberately.

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	A proxy cache is a shared cache. One stored copy of <code>logo.png</code> is served to Ravi, Sita and Anu alike. Right panel of Figure 3.2.
B	TRUE	Your hard refresh instructs your browser . It does not command the proxy sitting on the network, which may answer from its own stored copy. Figure 3.3.
C	TRUE	A browser cache hit needs no network journey at all ; a proxy hit still needs a round trip. Less distance means less latency, so browser level wins. Figure 3.1.
D	TRUE the answer	Since A, B and C are all true, "All of the above" is the correct single choice.

ANSWER TO QUESTION 3

Option D – All of the above

Matches your selection.

TRAP BOX • DO Q2 AND Q3 CONTRADICT EACH OTHER?

No, and it is worth being clear because the two questions sit side by side.

Q2 C says a hard refresh clears **your browser's** cache. True.

Q3 B says a hard refresh may **still** get a cached response **from a proxy**. Also true.

Two different caches, two different owners. Clearing yours does not empty theirs.

ETag, If-Match and If-None-Match

Caching raises an obvious worry: **how do I know my stored copy is still the current one?** The answer is a small piece of HTTP called the **ETag**.

STORY BOX • THE VERSION STICKER ON THE NOTICE

The notice board holds a notice with a sticker on it reading **v7**. You photograph it and go home, remembering "I have v7".

Next week, instead of copying the whole notice again, you phone the office and ask: **"is it still v7?"**

- "Yes, still v7" – a two-second call. You keep your photo. Nothing was re-copied.
- "No, it is v8 now" – they read you the whole new notice.

That sticker is the **ETag** (short for **entity tag**). It is the server's version label for one particular version of one resource.

DEFINITION BOX • ETag

An **ETag** is a **response header** the server attaches to a resource, identifying that exact version of it. It is a string of ASCII characters written **inside double quotes**, for example `ETag: "33a64df5"`. Usually it is a hash of the content, so any change to the file produces a different ETag.

4.1 The ETag round trip, drawn out

How an ETag saves you a whole download

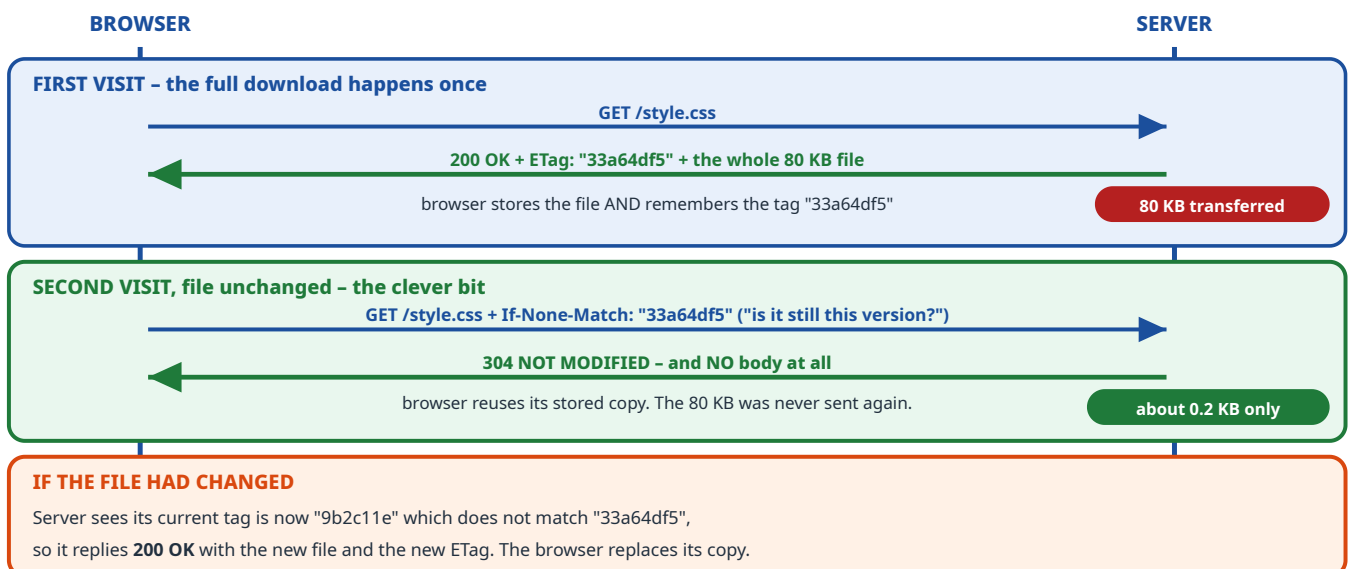


Figure 4.1 – This is called a **conditional request**: the response depends on whether the condition matched.

WHY 304 MATTERS SO MUCH

A **304 Not Modified** response carries **no body**. You still pay for one small round trip, but you avoid re-downloading the bytes. On a page with 40 images and stylesheets, that is the difference between reloading 2 MB and reloading almost nothing.

4.2 If-Match and If-None-Match are opposites

Both headers carry an ETag. The difference is the **condition** they set, and Question 9 turns on getting them the right way round.

Two conditional headers, two opposite conditions

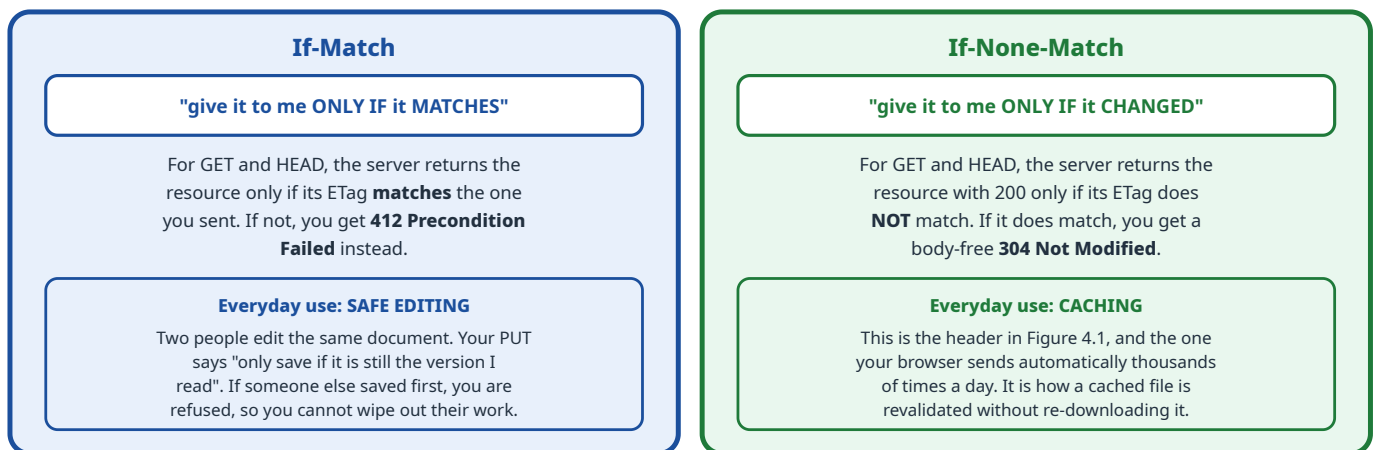


Figure 4.2 – Read the names literally: *If-Match* = if it matches. *If-None-Match* = if none of them match.

THE MEMORY TRICK

The header names say exactly what they mean, so trust them.

If-Match – "if it matches, proceed." Used when you are about to **write**.

If-None-Match – "if none match, send it." Used when you want to **reuse a cache**.

Question 9

1 mark • select all that apply

Which of the following is/are correct regarding the E-Tag and If-Match?

- ☒ A. E-Tag is a response header to validate the current version of resource.
- ☒ B. E-Tag is a string of ASCII characters placed between double quotes.
- ☒ C. For get and head request, If-Match request header is sent to get the resource only if E-Tag value matches.
- ☐ D. For get and head request, If-Match request header is sent to get the resource only if E-Tag value does not match.

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	Two facts in one sentence, both right. It is a response header (the server sends it), and its purpose is to identify and validate the current version of the resource.
B	TRUE	An ETag really is written as an ASCII string wrapped in double quotes: <code>ETag: "33a64df5"</code> . The quotes are part of the required format, not decoration. (A weak tag is prefixed with <code>W/</code> , as in <code>W/"33a64df5"</code> .)
C	TRUE	Correct, and this is exactly how the HTTP specification defines it: for GET and HEAD, the resource is returned only if the ETag matches one of the values in <code>If-Match</code> . If it does not match, the server answers 412 Precondition Failed . Left panel of Figure 4.2.
D	FALSE	This describes If-None-Match , not <code>If-Match</code> . C and D are direct opposites, so exactly one can be true – and the name <i>If-Match</i> tells you which.

ANSWER TO QUESTION 9

A, B and C

Matches your selection. Only D is false, because "does not match" belongs to `If-None-Match`.

The whole family of conditional headers, for the end term

Header	Direction	Meaning
ETag	response	"this is version <code>"abc"</code> of the resource"
If-None-Match	request	"send it only if the version is not <code>"abc"</code> " – otherwise 304. Used for caching.
If-Match	request	"proceed only if the version is <code>"abc"</code> " – otherwise 412. Used for safe writes.
Last-Modified	response	A date instead of a hash: "last changed at this time".
If-Modified-Since	request	The date-based cousin of <code>If-None-Match</code> .
Cache-Control	both	The rules: how long to cache (<code>max-age</code>), and whether a shared cache may store it (<code>public</code> vs <code>private</code> , <code>no-store</code>).

Flask-Caching: @cached vs @memoize

So far the caches have been outside our control. Now we cache **inside** our own Flask application, and there are two decorators that look interchangeable but are not.

5.1 The idea of a cache key

STORY BOX • THE LABELLED TIFFIN BOXES

Amma cooks and stores food in boxes on a shelf. To find food again she needs a **label** on each box. If every box is labelled just "**FOOD**", then the first box goes on the shelf and every later dish is assumed to be that same box – ask for rajma and you are handed the poha that got there first.

If instead the labels read "**FOOD - rajma**" and "**FOOD - poha**", everything works.

That label is the **cache key**. And the entire difference between the two decorators is **what goes on the label**.

DEFINITION BOX • THE TWO DECORATORS

- `@cache.cached()` builds its key from the **request path** (the default key is `view/<request.path>`). The function's **parameters are not part of the key**.
- `@cache.memoize()` builds its key from the **function plus its arguments**. Different arguments therefore get different cache entries.

What goes on the label?

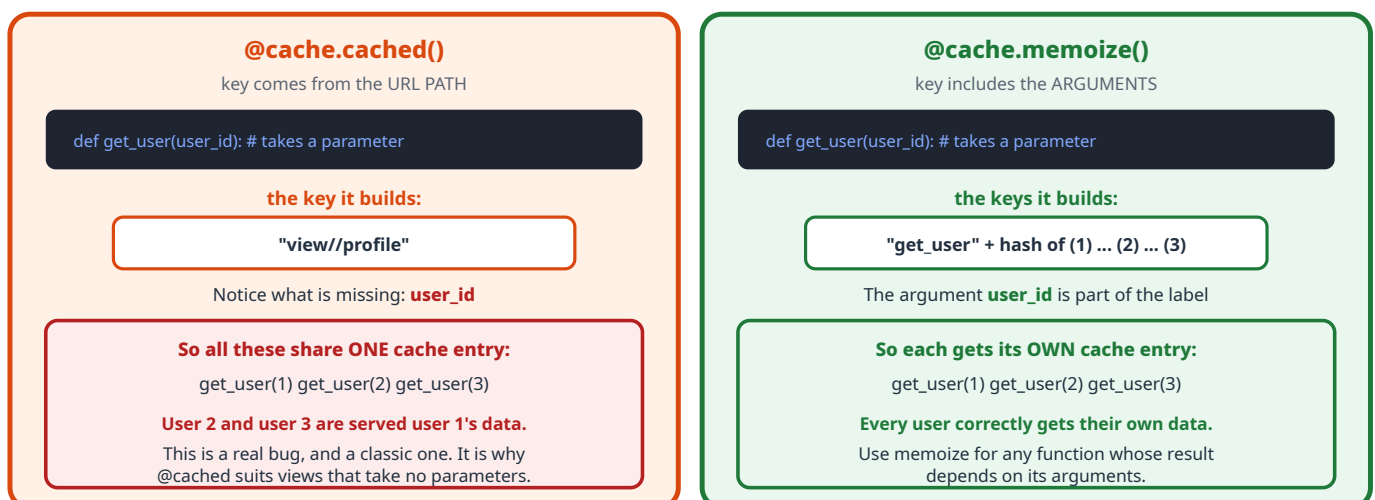


Figure 5.1 – The single difference: `memoize` puts the arguments in the key, `cached` does not.

5.2 Seeing the bug happen, step by step

```
# THE BUG: @cached on a function that takes a parameter
@cache.cached(timeout=60)
def square(n):
    return n * n

square(2) # cache miss -> computes 4, stores it under the path-based key
square(5) # cache HIT on the SAME key -> returns 4  WRONG, should be 25
```

```
# THE FIX: @memoize puts the argument in the key
@cache.memoize(timeout=60)
def square(n):
    return n * n

square(2) # miss -> 4 stored under square + (2)
square(5) # miss -> 25 stored under square + (5)  CORRECT
square(2) # hit -> 4 from cache  CORRECT and fast
```

TRAP BOX • THE DIRECTION STUDENTS REVERSE

It is very easy to remember "one of them includes the parameters" and then attach it to the wrong decorator. Anchor it with the word itself: **memoize** comes from *memoization*, which in computer science means **caching a function's result per set of arguments**. Arguments are baked into the very meaning of the word. So: **memoize = arguments matter**.

5.3 The special case: a function with no parameters

STORY BOX • ONLY ONE DISH ON THE MENU

The whole problem with the label **"FOOD"** was that several different dishes competed for it. But suppose Amma only ever cooks **one** dish. Now the label "FOOD" is perfectly unambiguous – and a more detailed label would make no practical difference, because there is nothing to distinguish.

Same with our decorators. Their only difference is whether arguments join the key. Take away the arguments and the difference disappears.

A function with NO parameters: the two decorators behave the same way

```
def get_homepage_stats(): # no parameters at all
```

with @cached

one key, one stored result

there are no arguments to leave out,
so nothing can collide
works correctly

with @memoize

one key, one stored result

it would include arguments, but
there are none to include
works correctly

Same number of entries, same result, same behaviour. This is Q1 option C.

Figure 5.2 – Remove the arguments and you remove the only thing that separated the two decorators.

Question 1

1 mark • select all that apply

Which of the following statements(s) is/are true regarding flask_caching?

- A. The cache decorator includes the function parameters in the cache key.
- B. The memoize decorator includes the function parameters in the cache key.
- C. Both the cache and memoize decorators work the same way for functions not having any parameters.
- D. All of the above

Option-by-option reasoning

Option	Verdict	Why
A	FALSE	The wrong way round. <code>@cached</code> builds its key from the request path (<code>view/<request.path></code>), not from the function parameters. That is precisely why calling it with different arguments returns the first stored answer to everybody. Left panel of Figure 5.1.
B	TRUE	Correct, and it is the definition of memoization: the arguments are part of the cache key, so each distinct set of arguments gets its own entry.
C	TRUE	The only difference between them is whether arguments enter the key. For a function with no arguments there is nothing to include or exclude, so both produce a single cache entry and behave identically. Figure 5.2.
D	FALSE	Requires A to be true, and A is false. Note also that A and B are near-opposites: spotting that pair immediately tells you D cannot work.

ANSWER TO QUESTION 1

B and C

Matches your selection. A is false because `@cached` keys on the request path, not the parameters.

THE PRACTICAL RULE FOR YOUR OWN PROJECT

- Caching a **view function** whose output depends only on the URL? Use `@cached` .
- Caching **any function whose result depends on its arguments**? Use `@memoize` .
- Both take a `timeout` in seconds. Clear entries with `cache.delete_memoized(fn, args)` when the underlying data changes.

Cookies: the hidden weight on every request

STORY BOX • THE IDENTITY CARD YOU MUST SHOW EVERY TIME

Imagine a building where you must show your ID card at the door **every single time** you walk in – and you enter 80 times a day. If the card is a small plastic chip, fine. If it is a fat 4 KB folder of documents, you have just added that weight to **all 80 entries**.

A **cookie** is that ID card. The browser attaches it automatically to **every** request for that domain – every image, every stylesheet, every icon. So cookie size is multiplied by the number of requests, which is why it affects performance.

DEFINITION BOX • COOKIE

A **cookie** is a small piece of data the server asks the browser to store and send back on subsequent requests to that domain, in the **Cookie request header**. It is how a site remembers you are logged in. Cookies are sent **automatically and on every matching request**, whether that request needs them or not.

One page, 50 requests. What does a 4 KB cookie cost?



This is a real, named optimisation: serve static assets from a COOKIE-FREE DOMAIN.

Your logo does not need to know who is logged in, so sending your session cookie with it is wasted bytes.

Figure 6.1 – Cookie size is multiplied by the number of requests, so it absolutely can impact performance.

6.1 The `domain` attribute, and why subdomains matter

STORY BOX • THE FAMILY SURNAME

If a rule is written for "**the Sharma family**", it applies to *every* Sharma in the house – parents, children, cousins. If it is written for "**Ravi Sharma only**", it applies to him alone.

Setting a cookie's `domain` attribute to `example.com` is the family version: the cookie then travels to **example.com and all its subdomains**, including `static.example.com`, `api.example.com` and `blog.example.com`.

Does a request to static.example.com carry cookies set for example.com?

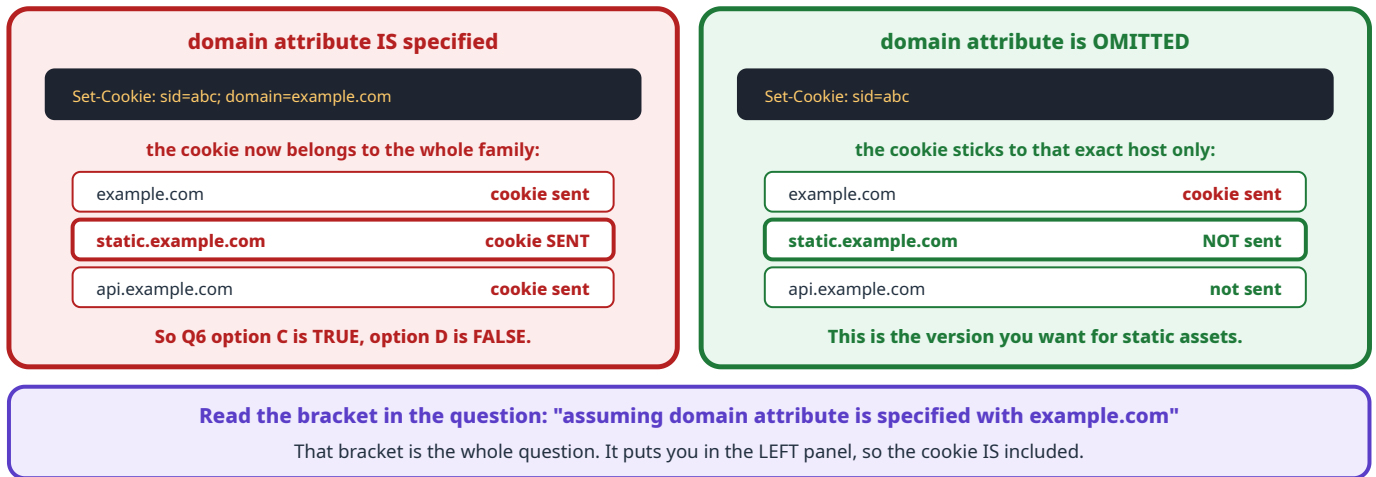


Figure 6.2 – Specifying `domain` widens a cookie's reach to subdomains. Omitting it keeps the cookie to one host.

Question 6

1 mark • select all that apply

Which of the following is the correct statement?

- A. Cookie size can impact the website performance.
- B. Cookie size does not affect the website performance.
- C. A request to static.example.com will include cookies set for example.com (assuming domain attribute is specified with "example.com").
- D. A request to static.example.com will not include cookies set for example.com (assuming domain attribute is specified with "example.com").

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	Cookies ride along on every request to the domain, so their size is multiplied by the number of requests. On a 50-request page a 4 KB cookie becomes 200 KB of pure overhead (Figure 6.1). Upload bandwidth is usually much slower than download, which makes this worse than it looks.
B	FALSE	Direct opposite of A, so exactly one of the pair is correct.
C	TRUE	Because the <code>domain</code> attribute was explicitly set to <code>example.com</code> , the cookie applies to that domain and its subdomains – so <code>static.example.com</code> receives it. Left panel of Figure 6.2.
D	FALSE	Opposite of C. It would be true only if the <code>domain</code> attribute had been omitted , which the question explicitly rules out.

ANSWER TO QUESTION 6

A and C

Matches your selection.

TRAP BOX • THE COUNTER-INTUITIVE BIT

Many students assume that naming a domain **restricts** the cookie. It does the opposite: specifying `domain=example.com` **broadens** it to every subdomain. If you want a cookie confined to one host, **leave the attribute out altogether**.

DOMContentLoaded VS load

STORY BOX • THE WEDDING HALL

You have to move the chairs in a wedding hall. Two moments you could wait for:

Moment 1: the hall is built, the floor is laid and all the furniture is in place. The chairs exist, so you can start moving them *now*.

Moment 2: everything above, *plus* the flowers have arrived, the lights are hung and the photographs are on the walls. Much later.

Moment 1 is **DOMContentLoaded**. Moment 2 is **load**. If your job is only to move a chair, waiting for the flowers is a waste of time.

DEFINITION BOX • THE TWO EVENTS

- **DOMContentLoaded** fires as soon as the HTML has been parsed and the **DOM tree is built**. It does **not** wait for images or other subresources.
- **load** fires only when the page and **all** its dependent resources – images, stylesheets, fonts, iframes – have finished loading.

DOMContentLoaded **always fires first**, often by a large margin.

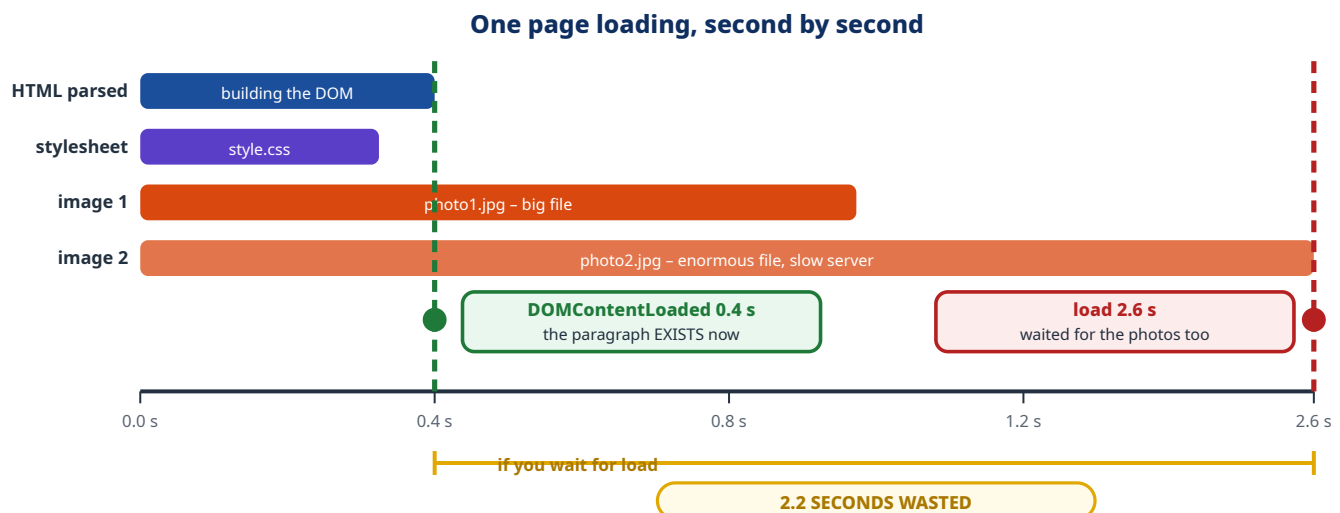


Figure 7.1 – Updating a paragraph only needs the DOM. Waiting for **load** means waiting for photographs you never touch.

Question 7

1 mark • single correct

Suppose you have a web page which includes a paragraph with some content, images and a stylesheet. You want to update the content of the paragraph with JavaScript. Which of the event, among `load` and `DOMContentLoaded`, you should wait to fire before updating the content for better performance?

- A. `load`
- B. `DOMContentLoaded`
- C. Both of them will result in same performance.
- D. DOM is not required to be loaded fully before updating the content.

Step-by-step reasoning

1 What does the job actually need?

You want to change the text inside a `<p>` element. To do that, that element must **exist in the DOM**. That is the only requirement.

2 Which event guarantees that, and nothing more?

`DOMContentLoaded`. It fires the moment the HTML is parsed and the DOM is built. The paragraph is present and editable from that instant.

3 What would waiting for `load` add?

It would additionally wait for every image and stylesheet to finish downloading. Your paragraph text has nothing to do with the photographs, so that waiting is pure delay – 2.2 seconds of it in Figure 7.1.

4 Conclusion.

Wait for the earliest event that satisfies your requirement. That is `DOMContentLoaded`, so the user sees the updated text sooner.

Option-by-option reasoning

Option	Verdict	Why
A	FALSE	<code>load</code> works, but it is slower . The question asks for better performance, so "it works" is not enough.
B	TRUE the answer	Fires as soon as the DOM is ready, which is all the task needs. Earliest safe moment means best performance.
C	FALSE	They differ, often by seconds, because <code>load</code> waits for images. Figure 7.1 shows the gap.
D	FALSE	You cannot edit an element that does not exist yet. Running the script before the DOM is built gives you <code>null</code> and a JavaScript error – the classic "Cannot set property of null" mistake.

ANSWER TO QUESTION 7

Option B – DOMContentLoaded

Matches your selection.

```
// the right way: act as soon as the DOM exists
document.addEventListener('DOMContentLoaded', () => {
  document.querySelector('p').textContent = 'updated text'
})
```

```
// slower: this waits for every image to download first
window.addEventListener('load', () => { /* ... */ })
```

WHEN YOU GENUINELY DO NEED LOAD

Use `load` only when your code depends on a resource beyond the DOM – for example measuring an image's real rendered height, or working with a fully painted canvas. For everything else, `DOMContentLoaded` is the right choice.

Images: the biggest easy win

STORY BOX • THE GIANT POSTER AND THE SMALL FRAME

You need a photograph to fit a small 10 cm frame. Two ways:

The wasteful way: order a huge 20 cm poster, carry it home, then fold it down to fit the frame. You paid for paper you threw away, you carried extra weight, and you did extra work folding it.

The sensible way: order a 10 cm print. Less to carry, nothing wasted, no folding.

Setting `width="100"` on a 200 px image is the folding. The browser still **downloads every byte** of the big image, then does extra work shrinking it for display.

You need a 100 x 100 image on the page. Two ways to get it.



Figure 8.1 – Doubling each side gives **four times** the pixels. Scaling in HTML hides the size on screen but not on the wire.

THE ARITHMETIC WORTH REMEMBERING

A 200 x 200 image holds **40 000** pixels. A 100 x 100 image holds **10 000**. That is **four times** as much pixel data, not twice – because both dimensions doubled. This is why oversized images are usually the single heaviest thing on a slow web page.

Question 8

1 mark • single correct

Suppose you want to embed an image of 100*100 px in your webpage. Which of the following statements is correct?

- A. Using an image with dimension 200*200 px and scaling it using HTML will perform better than using a 100*100 px image.
- B. Using an image with dimension 200*200 px and scaling it using HTML will perform worse than using a 100*100 px image.
- C. Scaling using HTML does not impact the performance.
- D. None of the above

Option-by-option reasoning

Option	Verdict	Why
A	FALSE	Backwards. Downloading four times the pixel data and then doing extra work to shrink it cannot be <i>faster</i> than downloading the right size in the first place.
B	TRUE the answer	Correct on both counts: a bigger file to transfer , plus extra CPU and memory for the browser to resample it down to 100 px. Left panel of Figure 8.1.
C	FALSE	It certainly does impact performance – that is the entire point of the "serve correctly sized images" recommendation that Lighthouse gives (Part 2).
D	FALSE	Since B is correct, "None of the above" cannot be.

ANSWER TO QUESTION 8

Option B

Scaling a larger image down in HTML performs worse. Matches your selection.

THE ONE EXCEPTION, FOR COMPLETENESS

On high-density screens (Retina and most modern phones) it is deliberate practice to serve an image about twice the CSS size so it looks sharp. That is a **quality** decision, knowingly paid for with bytes. It does not change the answer here: the question is purely about performance, and on that measure the smaller correctly-sized file always wins. The modern way to serve both is the `srcset` attribute.

CDN: a copy near every user

STORY BOX • ONE FACTORY, OR SHOPS IN EVERY TOWN

A biscuit company has one factory in Chennai. If every customer in India had to travel to Chennai for a packet of biscuits, buying biscuits would be slow and the factory gate would be permanently jammed.

So the company opens **shops in every town**. Same biscuits, made in the same factory, but now a customer in Jaipur walks 5 minutes instead of travelling 2000 km.

That network of shops is a **CDN**. The factory is your **origin server**; each shop is an **edge server**. Nothing about the content changes – only the **distance** does.

DEFINITION BOX • CDN

CDN stands for **Content Delivery Network**. It is a geographically distributed **network of servers** that keeps cached copies of your content close to users, so content is delivered faster. The nearby servers are called **edge servers** or **points of presence**.

Without a CDN, and with one

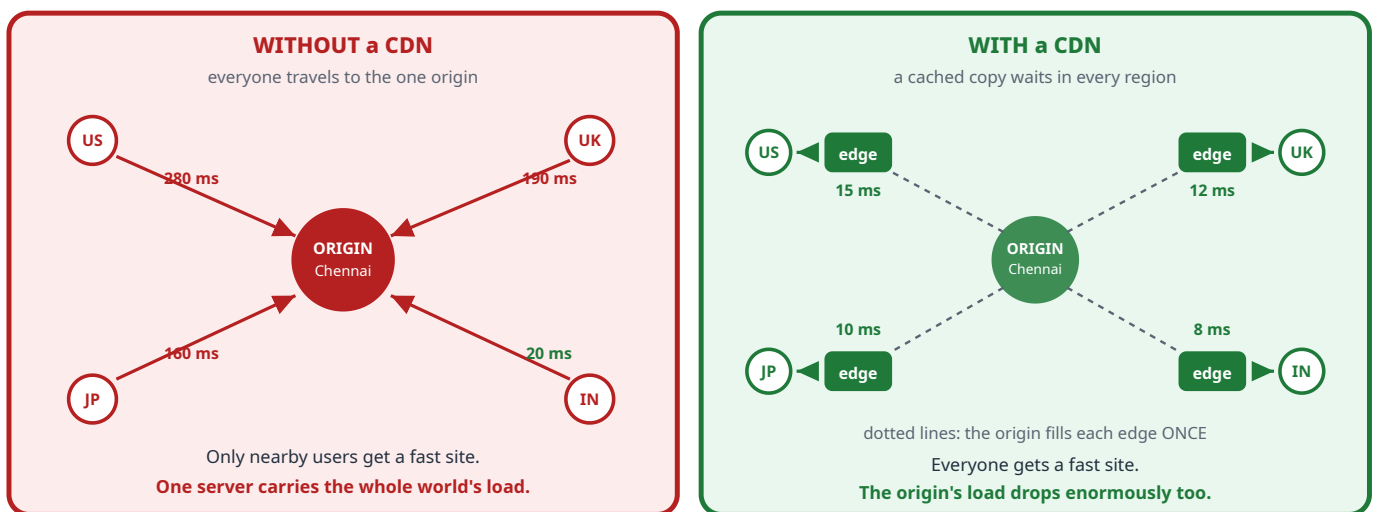


Figure 9.1 – A CDN reduces the **distance** between the client and the resource, which is why delivery gets faster.

Question 10

1 mark • select all that apply

Which of the following is/are true regarding CDN?

- A. CDN is an acronym for content delivery network.
- B. It is the network of servers to provide fast delivery of web content.
- C. It reduces the distance between client and the resource, which results in faster delivery of content.
- D. None of the above

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	Simply the expansion of the abbreviation: C ontent D elivery N etwork.
B	TRUE	The word network is doing real work in that sentence. A CDN is not one server but many, spread across regions, all serving copies of your content.
C	TRUE	This states the actual mechanism, and it is the reason a CDN works: shorter physical distance means fewer network hops and lower latency. Figure 9.1.
D	FALSE	"None of the above" fails because A, B and C are all true.

ANSWER TO QUESTION 10

A, B and C

Matches your selection. Note there was no "All of the above" option here – you had to tick the three individually.

TRAP BOX • "NONE OF THE ABOVE" VS "ALL OF THE ABOVE"

This paper uses both. In Q3 the right answer was **All of the above**; here the last option is **None of the above** and is wrong. Never answer from the shape of the option list. Read every statement, decide true or false for each, and only then look at the last option.

Answer key, cheat sheet and practice

10.1 Master answer key

Q	Topic	Answer	The one line that decides it
1	flask_caching decorators	B, C	memoize keys on arguments; cached keys on the request path. With no arguments they match.
2	Caching levels	A, C	Browser caching exists, proxy caching exists too, and a hard refresh clears your own cache.
3	Proxy vs browser cache	D	All three are true: proxies are shared, hard refresh cannot reach them, browser cache is nearest.
4	Performance criteria	A, B	Throughput-with-latency and scalability. A framework choice is not a measurement.
5	Performance tools	A, C	GTmetrix and Lighthouse analyse pages. Postman tests APIs.
6	Cookies	A, C	Cookie size costs bytes on every request, and domain=example.com reaches subdomains.
7	DOM events	B	Editing a paragraph needs only the DOM, so use the earlier event.
8	Image scaling	B	200x200 is four times the pixels, plus resizing work. Worse, not better.
9	ETag and If-Match	A, B, C	Response header, quoted ASCII string, and If-Match means "only if it matches".
10	CDN	A, B, C	Content Delivery Network, a network of servers, shortening client-to-resource distance.

YOUR ATTEMPT: 10 OUT OF 10

Every option you selected matches this key. Nothing to change. The value of the rest of this part is making sure you can rebuild the reasoning when the end-term paper rewords these ideas.

10.2 The whole week on one page

Every performance lever in this paper, grouped by where it acts

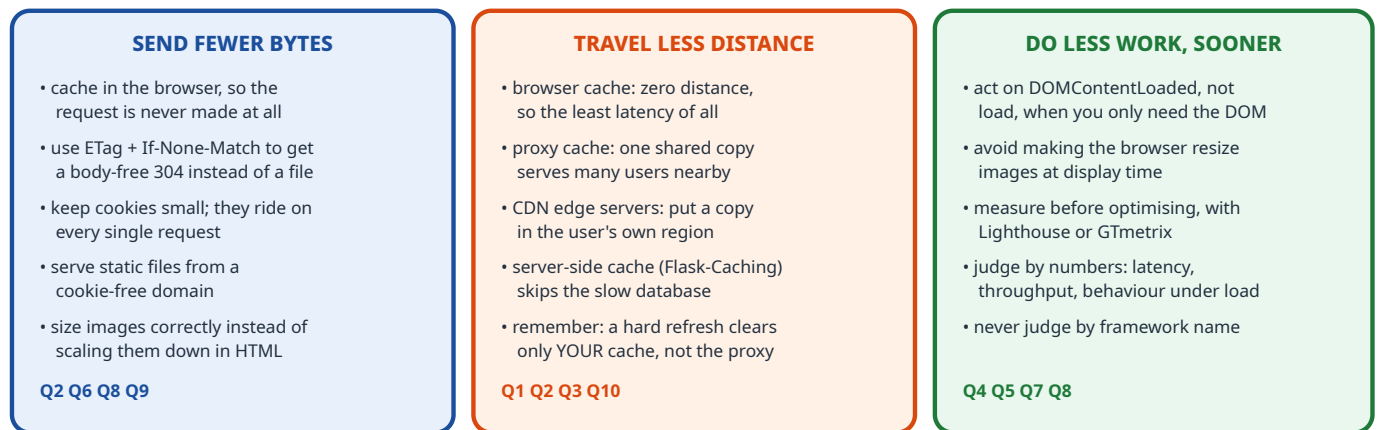


Figure 10.1 – Three questions to ask of any slow page: can I send fewer bytes, travel less far, or do less work?

10.3 The one-page cheat sheet

Measuring performance

- **Latency:** time for one request. A duration. Lower is better.
- **Throughput:** requests served per unit time. A rate. Higher is better.
- **Scalability:** whether both hold up as load grows.
- A framework choice is **not** a performance criterion. If you cannot put a number on it, it does not count.
- Tools: **Lighthouse**, **GTmetrix**, WebPageTest, PageSpeed Insights, DevTools. **Postman is for API testing**, not page performance.

Caching

- Four levels: **browser** (private), **proxy** (shared), **CDN edge** (shared, geographic), **server-side** (Flask-Caching + Redis).
- **Browser level has the least latency** – no network journey at all.
- A **proxy cache serves its copy to multiple users**. That is what "shared" means.
- **Hard refresh** clears **your browser's** cache for that page. It has no power over a proxy, which may still answer from its own copy.
- **Cache hit** = served from cache. **Cache miss** = had to go to the origin.

ETag and conditional requests

- **ETag**: a **response** header identifying a version of a resource; an ASCII string in **double quotes**, e.g. "33a64df5" .
- **If-None-Match**: "send it only if it has **changed**". Match means **304 Not Modified** with no body. This is the caching workhorse.
- **If-Match**: "proceed only if it **matches**". No match means **412 Precondition Failed**. Used for safe writes.
- Date-based cousins: `Last-Modified` and `If-Modified-Since` .

Flask-Caching

- `@cached` : key is the **request path** (`view/<path>`). Parameters are **not** in the key.
- `@memoize` : key includes the **function arguments**. Use it for any function whose result depends on its inputs.
- For a function with **no parameters**, the two behave **identically**.
- Using `@cached` on a parameterised function is a real bug: every argument collides on one entry.

Front-end details

- **Cookies** are attached to **every** request for the domain, so size matters. Fix: a **cookie-free domain** for static assets.
- `domain=example.com` makes the cookie reach **example.com and all subdomains**, including `static.example.com` . **Omitting** the attribute keeps it to that one host.
- **DOMContentLoaded** : DOM is built. Fires **first**. **load** : everything including images is done. Fires later.
- Only need the DOM? Use `DOMContentLoaded` .
- **Never scale images down in HTML**. 200x200 is **four times** the pixels of 100x100, plus resizing work.

CDN

- **Content Delivery Network**: a distributed **network of servers** holding cached copies.
- It works by **reducing the distance** between client and resource.
- Side benefit: the origin server's load drops sharply.
- The nearby servers are **edge servers** or **points of presence**.

10.4 Exam technique: the keyword radar

Word you spot	What it usually means	Where it happened here
cannot, does not	likely FALSE	Q2 option B ("caching cannot be done at proxy level"), Q6 option B, Q8 option C.
All of the above	check every option	A trap in Q1, Q2, Q4 and Q5 – but the correct answer in Q3. Never assume.
None of the above	usually FALSE	Q8 option D and Q10 option D, both wrong.
opposite pairs	exactly one is right	Q1 A vs B, Q6 A vs B and C vs D, Q9 C vs D, Q8 A vs B.
a bracketed condition	that IS the question	Q6: "(assuming domain attribute is specified with example.com)" decides C and D entirely.
a familiar tool name	be suspicious	Q4 option C mentions Flask and Django; Q5 option B mentions Postman. Both wrong.
"better performance"	not just "does it work"	Q7: <code>load</code> works, but the question asks which is <i>better</i> .

10.5 Practice questions

Cover the answers. If you can do these eight, you have genuinely learned Week 11.

P1. You write `@cache.cached(timeout=60)` above `def get_marks(student_id)`. Ravi (id 1) loads his marks, then Sita (id 2) loads hers. What does Sita see, and why? What is the fix?

P2. A colleague says "I hard refreshed and I still see the old logo, so caching is broken." Give two possible explanations.

P3. Your page makes 60 requests to `myshop.com` and your session cookie is 2 KB. How many bytes of cookie overhead per page load? How would you reduce it?

P4. A server responds `304 Not Modified`. What did the browser send to cause this, and roughly how many bytes of body came back?

P5. Which is faster to serve and why: a resource cached in your browser, or the same resource cached at your ISP's proxy?

P6. You need JavaScript to read the final rendered **height in pixels** of a photo on the page. Should you use `DOMContentLoaded` or `load`? Why is this different from Question 7?

P7. You set `Set-Cookie: sid=xyz; domain=example.com`. List which of these receive the cookie: `example.com`, `static.example.com`, `example.org`, `a.b.example.com`.

P8. An image is displayed at 300x300. You have a 900x900 file. Roughly how many times more pixel data are you sending than necessary?

ANSWERS TO THE PRACTICE QUESTIONS

P1. Sita sees **Ravi's marks**. `@cached` keys on the request path, not on `student_id`, so both calls collide on one cache entry and the first stored result is returned to everyone. The fix is `@cache.memoize(timeout=60)`, which puts `student_id` into the key. (This is also a security bug, not merely a performance one.)

P2. (1) A **proxy cache** between them and the server still holds the old file – their refresh only cleared their own browser. (2) The logo is referenced by an **unchanged URL** and is being served from a **CDN edge** that has not yet expired its copy. A third possibility: they refreshed a different page from the one that embeds the logo. The standard cure is **cache busting** – rename the file to `logo.v2.png` or add `?v=2`.

P3. $60 \times 2 \text{ KB} = 120 \text{ KB}$ of cookie sent per page load, all of it upload traffic. Reduce it by serving static assets from a **cookie-free domain** (or a CDN hostname), and by storing only a small session id in the cookie rather than user data.

P4. The browser sent a **conditional request** with `If-None-Match: "<etag>"` (or `If-Modified-Since`). The 304 carries **no body at all** – only headers, a few hundred bytes. The browser then reuses its stored copy.

P5. The **browser** copy. It is read from local disk with **no network round trip whatsoever**, whereas the proxy still needs a request to leave your machine and come back. Zero distance beats short distance.

P6. load. Reading a photo's rendered height requires the image to have actually downloaded, because until then the browser does not know its dimensions. Question 7 only edited **text**, which needs nothing but the DOM. The rule is: wait for the earliest event that *genuinely* satisfies your dependency.

P7. Receive it: `example.com`, `static.example.com`, and `a.b.example.com` (all subdomains at any depth). **Does not:** `example.org` – a completely different domain.

P8. $900 \times 900 = 810\,000$ pixels; $300 \times 300 = 90\,000$ pixels. That is **9 times** the pixel data (because 3×3), all downloaded and then thrown away during resizing.

10.6 Glossary

Term	Meaning in one sentence
Latency	The time taken to serve one request; a duration.
Throughput	How many requests are served per unit of time; a rate.
Scalability	Whether performance holds up as the load grows.
Load	How many users or requests are hitting the system at once.
Lighthouse	Google's page auditing tool, built into Chrome DevTools.
GTmetrix	An online service that times your page and charts every request.

Waterfall chart	A diagram showing when each request started and finished.
Cache	A nearby stored copy of something that lives further away.
Cache hit / miss	Served from the cache / had to fetch from the origin.
Origin server	The authoritative source of the content.
Private cache	A cache serving one user, e.g. the browser cache.
Shared cache	A cache serving many users, e.g. a proxy or CDN edge.
Proxy server	A machine between client and server that can cache responses.
Hard refresh	Reloading while bypassing and clearing your own browser cache.
Stale	A cached copy that no longer matches the origin.
Cache busting	Changing a file's URL so caches are forced to fetch it again.
ETag	A response header naming the current version of a resource.
Conditional request	A request whose response depends on a precondition being met.
304 Not Modified	"Your copy is current" – a response with no body.
412 Precondition Failed	"Your condition did not hold" – the If-Match failure.
Cache-Control	The header carrying caching rules such as <code>max-age</code> and <code>private</code> .
Cache key	The label under which a cached value is stored and looked up.
Memoization	Caching a function's result per set of arguments.
Cookie	Small data stored by the browser and resent on every matching request.
Cookie-free domain	A separate hostname for static files, so no cookies are attached.
DOM	The browser's in-memory tree of the page's elements.
DOMContentLoaded	Event fired once the HTML is parsed and the DOM is built.
load	Event fired once the page and all subresources have finished loading.
CDN	Content Delivery Network: distributed servers holding cached copies.
Edge server	A CDN server placed geographically close to users.

10.7 Final word

IF YOU REMEMBER ONLY SEVEN SENTENCES FROM THIS WHOLE BOOK

1. Performance is **numbers**: latency, throughput, scalability. A framework name is not a number.
2. **Lighthouse and GTmetrix** measure pages; **Postman** tests APIs.
3. Caches live at **browser, proxy, CDN and server** level, and **browser level has the least latency**.
4. A **hard refresh clears only your own cache** – a proxy can still serve you a stored copy.
5. **ETag + If-None-Match** gets you a body-free **304**; **If-Match** means "only if it matches" and fails with **412**.
6. **memoize keys on arguments, cached keys on the path**, and with no arguments they are the same.
7. Small cookies, correctly sized images, **DOMContentLoaded** over **load**, and a **CDN** to shorten the distance.

Prepared as a step-by-step teaching solution for Week 11.

Work through the figures with a pencil in hand; that is where the learning actually happens.

NOTES ON SOURCES

Technical facts were checked against the official documentation for each technology: [the Flask-Caching documentation](#) for the behaviour of `@cached` (default key `view/request.path`) and `@memoize` (arguments included in the cache key); [MDN on If-Match](#) and [MDN on If-None-Match](#) for conditional requests, which confirm that for GET and HEAD the resource is returned only when the If-Match ETag matches, and that a mismatch yields 412 Precondition Failed; MDN on [DOMContentLoaded](#) and [HTTP caching](#); and MDN on [cookies](#) for the behaviour of the `domain` attribute with subdomains. All explanations, analogies and diagrams here were written and drawn for this document. Content was rephrased for compliance with licensing restrictions.